task runs, the result will be captured in the future, as will any exception thrown.

Since you've been using futures, you don't have a result array, so you must store the result from the final block in a variable ❼ rather than in a slot in the array. Also, because you have to get the values out of the futures, it's now simpler to use a basic `for` loop rather than `std::accumulate`, starting with the supplied initial value ❽ and adding in the result from each future ❾. If the corresponding task threw an exception, this will have been captured in the future and will now be thrown again by the call to `get()`. Finally, you add the result from the last block ❿ before returning the overall result to the caller.

So, that's removed one of the potential problems: exceptions thrown in the worker threads are rethrown in the main thread. If more than one of the worker threads throws an exception, only one will be propagated, but that's not too big a deal. If it really matters, you can use something like `std::nested_exception` to capture all the exceptions and throw that instead.

The remaining problem is the leaking threads if an exception is thrown between when you spawn the first thread and when you've joined with them all. The simplest solution is just to catch any exceptions, join with the threads that are still `joinable()`, and rethrow the exception:

```
try
{
     for(unsigned long i=0;i<(num_threads-1);++i)
     {
        // ... as before
     }
     T last_result=accumulate_block()(block_start,last);

     std::for_each(threads.begin(),threads.end(),
        std::mem_fn(&std::thread::join));
}
catch(...)
{
    for(unsigned long i=0;i<(num_thread-1);++i)
    {
        if(threads[i].joinable())
```